Relatório de Auditoria de Segurança

Observatório de Taxas

Data da auditoria: 31/08/2026

Stack: site estático (GitHub Pages) + coletores Python + GitHub Actions

Escopo auditado

Repositório iosbilario/observatorio-taxas, branch desliga-radar (HEAD 4e5de0b), auditado em 31/08/2026. Cobertura: scripts Python de coleta e geração (scripts/*.py, 10 arquivos), workflow de CI (.github/workflows/monitor.yml, dependabot.yml), configuração (config.yml, requirements.txt, .claude/*), todas as páginas HTML/JS servidas em docs/ (landing, embed, var, rebobinar, retransmissora, admin, correcao/*, reajuste/*, radar/* + radar.js), robots.txt, SECURITY.md e o histórico git completo (varredura de padrões de segredo em todas as revisões).

Nota metodológica

Cada uma das cinco categorias do protocolo foi mapeada para o equivalente desta stack antes da varredura (detalhe na página 2). Onde a categoria não se aplica — não há banco de dados, backend nem autenticação — isso é dito explicitamente, em vez de forçar achados.

Como cada categoria foi mapeada para esta stack

1. Banco sem tranca (isolamento de inquilino/dono)

A stack não tem banco de dados, backend nem sessões: é um site 100% estático no GitHub Pages, alimentado por robô (GitHub Actions) que commita JSONs públicos. Não existe mecanismo de isolamento a auditar — categoria NÃO APLICÁVEL. O SECURITY.md já prescreve RLS insert-only para o futuro backend Supabase (palpite.html), registrado como ponto forte.

2. Permissão definida no navegador

Sem servidor, não há endpoint privilegiado a cruzar com a UI. O equivalente na stack é o painel /admin/: página 'interna' protegida apenas por obscuridade (noindex + robots.txt). Foi auditado o que ela expõe e como.

3. IDOR

Não há rotas de backend nem objetos por dono: todos os handlers de 'dados' são fetches de JSON estático público por desenho (dados oficiais do BACEN). Percorridos todos os pontos de fetch do frontend — nenhum acessa recurso sensível parametrizado. Categoria NÃO APLICÁVEL.

4. Chaves expostas (hardcode)

Mapeado para: código-fonte, config.yml, workflow do Actions, .claude/, docs/ servido (bundle do 'frontend') e histórico git completo (git grep em todas as revisões por sk-ant-, ghp_, AKIA, chaves privadas, etc.). Auditados também os defaults e o manuseio de secrets no CI.

5. Inputs sem tratamento (XSS)

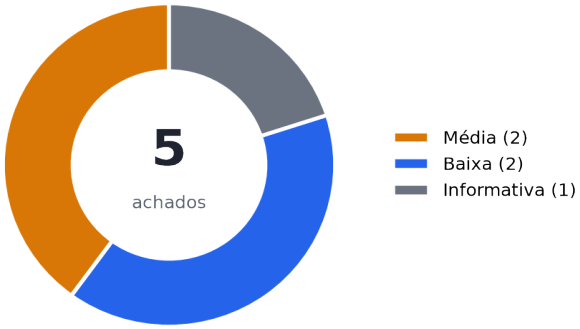
Stack: JavaScript vanilla com innerHTML — todos os 100+ pontos de innerHTML foram enumerados e cada interpolação rastreada até a origem (URLSearchParams, location.hash, JSONs de dados, API do GoatCounter). Verificada a função esc() de cada página, as allowlists de parâmetros e as CSPs por página.

Resumo executivo

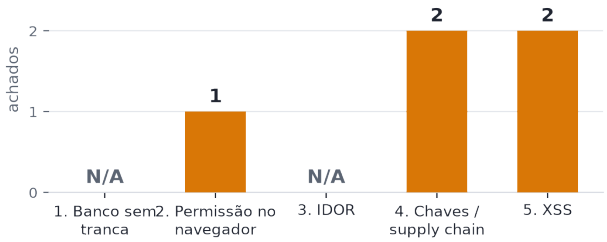
Nenhum achado crítico ou alto. O projeto trata segurança como parte do produto — escape de saída consistente, allowlists de parâmetros, CSP por página, Actions pinadas por SHA, zero segredos no código e no histórico git — e os cinco achados são de endurecimento: dois médios (validação de esquema de URL num componente ainda não ativado; pinagem de dependências do CI), dois baixos (escopo de secrets; CSP com 'unsafe-inline') e um informativo (painel /admin/ por obscuridade, decisão já documentada no próprio código).

0	0	2	2	1	9
CRÍTICA	ALTA	MÉDIA	BAIXA	INFORMATIVA	PONTOS FORTES

Achados por severidade



Achados por categoria



Categorias 1 (isolamento de inquilino) e 3 (IDOR) não se aplicam: não há banco de dados, backend nem objetos por dono — todos os dados servidos são públicos por desenho (séries oficiais do BACEN).

Pontos fortes — o que está protegido

- ✓ **Escape de saída consistente (anti-XSS).** Todos os ~100 usos de innerHTML interpolando dado dinâmico passam por esc() (&<>'") ou recebem apenas números formatados — verificado página a página (index, embed, var, rebobinar, admin, correcao/*, reajuste/*, radar/*).
- ✓ **Allowlist rigorosa de parâmetros de URL.** var.html:295-309 valida cada parâmetro (s contra códigos conhecidos, op/cmp por enum, datas por regex, sha por ^[0-9a-f]{7,40}$) e DESCARTA o inválido. embed.html:122-128 idem (Number(), regex aaaa-mm, enum de tema). sonda.html usa o slug só em fetch same-origin e textContent.
- ✓ **Dados de terceiros escapados no painel admin.** Os paths vindos da API do GoatCounter (manipuláveis por qualquer visitante que forje pageviews) são renderizados com esc(pathCurto(...)) em docs/admin/index.html:178 — o vetor de data poisoning → XSS está fechado.
- ✓ **Nenhum segredo hardcoded — código, configs e histórico git.** git grep em TODAS as revisões por padrões de chave (sk-ant-, ghp_, github_pat_, AKIA, xox*, BEGIN PRIVATE KEY): zero ocorrências. Secrets só via GitHub Actions secrets; goatcounter_code em config.yml é identificador público por desenho, documentado como tal.
- ✓ **Supply chain do frontend e do CI acima da média.** Actions pinadas por SHA de commit (monitor.yml:35,39); Chart.js pinado por versão exata com SRI + crossorigin (admin/index.html:32-34); Dependabot semanal para pip e github-actions; fontes self-hosted.
- ✓ **Parsing seguro nos coletores Python.** yaml.safe_load em todos os pontos (fetch.py:52, fetch_stats.py:62, builders); HTTPS em todas as chamadas; timeouts; falha de série não derruba o job; fetch_stats.py não sobrescreve stats.json bom quando a API falha.
- ✓ **CSP por página + higiene de links.** Toda página define CSP via meta (default-src 'self', object-src 'none', base-uri 'self'), referrer strict-origin-when-cross-origin, guard anti-frame (exceto embed.html, embutível por produto e documentado), e rel="noopener noreferrer" em todo target=_blank.
- ✓ **SECURITY.md vivo e correto.** Política publicada com regras vigentes (escape, allowlist, CSP, supply chain), canal de reporte + .well-known/security.txt, e regras já escritas para o futuro backend Supabase: service key só no Actions, RLS insert-only para anon, CHECK de faixa no banco.
- ✓ **Admin sem dado pessoal.** O pipeline de estatísticas só grava agregados (totais, grupos, top páginas, série diária) — nada de IP, user-agent ou identificador de visitante.

Pontos fracos — os riscos centrais

1. Supply chain do CI é o único caminho realista até os secrets. Dependências Python sem pin num job com escrita no repo e chaves no ambiente (A2+A3): é o par a fechar primeiro.

2. O contrato de dados do Radar ainda não impõe URL http(s). Quando o coletor real ligar, conteúdo de modelos de IA vira href sem validação de esquema (A1) — fechar antes de ativar.

3. A defesa anti-XSS depende de disciplina, não de política. O esc() hoje cobre tudo, mas com 'unsafe-inline' na CSP (A4) um único ponto esquecido no futuro já é explorável.

Achados detalhados

Sev.	Arquivo:linha	Achado
MÉDIA	`docs/radar/radar.js:352`	**A1 — href de fontes citadas aceita URL javascript:** 5. XSS
MÉDIA	`requirements.txt:1-3` `.github/workflows/monitor.yml:44,50`	**A2 — Dependências sem pinagem instaladas em job com permissão de escrita** 4. Chaves/supply chain
BAIXA	`.github/workflows/monitor.yml:29-31`	**A3 — Secrets expostos como env do job inteiro** 4. Chaves/supply chain

BAIXA	`docs/index.html:8`
	`docs/admin/index.html:8`
	`docs/embed.html:10`
	`demais páginas`
		A4 — CSP com script-src 'unsafe-inline' em todas as páginas
		5. XSS (defesa em profundidade)

INFORMATIVA	`docs/admin/index.html` `docs/robots.txt:3`	**A5 — Painel /admin/ público, protegido só por obscuridade (decisão documentada)** 2. Permissão no navegador

MÉDIA

A1 — href de fontes citadas aceita URL javascript:

`docs/radar/radar.js:352` · 5. XSS

```
return '<li><a href="' + esc(f.url) + '" target="_blank" ...
```

Por que é explorável: fontesHTML() monta links com esc(f.url). esc() escapa &<>'" — impede quebrar o atributo — mas não valida o ESQUEMA da URL: um valor javascript:... vira link executável ao clique (a CSP das páginas do Radar tem 'unsafe-inline', que também libera URLs javascript:). O campo fontes_citadas dos recibos virá de respostas de modelos de IA coletadas pelo futuro coletor — conteúdo de terceiros, influenciável por quem otimiza páginas para ser citado por IA.

Condição de explorabilidade: Hoje os recibos são fixtures commitadas (exemplo.invalid) e o Radar está fora do menu/sitemap — não explorável no estado atual. Torna-se explorável assim que o coletor real passar a escrever recibos com fontes vindas dos modelos.

MÉDIA

A2 — Dependências sem pinagem instaladas em job com permissão de escrita

`requirements.txt:1-3` · `.github/workflows/monitor.yml:44,50` · 4. Chaves/supply chain

```
requests
pyyaml
pillow (sem versão nem hash)
```

Por que é explorável: O job do cron instala requests, pyyaml, pillow (e anthropic, quando há chave) sempre na última versão publicada, sem pin nem --require-hashes. O mesmo job tem permissions: contents: write e os secrets no ambiente: uma release maliciosa no PyPI executaria com poder de commitar no repo e ler ANTHROPIC_API_KEY / GOATCOUNTER_TOKEN. Contrasta com o zelo do resto do projeto (Actions pinadas por SHA, SRI no CDN). O Dependabot está configurado para pip, mas sem versões pinadas ele não tem o que atualizar.

Condição de explorabilidade: Requer comprometimento de pacote no PyPI (supply chain) — probabilidade baixa, impacto alto.

BAIXA

A3 — Secrets expostos como env do job inteiro

`.github/workflows/monitor.yml:29-31` · 4. Chaves/supply chain

```
env:
ANTHROPIC_API_KEY: ${{ secrets.ANTHROPIC_API_KEY }}
GOATCOUNTER_TOKEN: ${{ secrets.GOATCOUNTER_TOKEN }}
```

Por que é explorável: As duas chaves ficam no ambiente de TODOS os steps — inclusive pip install e os geradores de página que não precisam delas. Qualquer código executado em qualquer step (ver A2) consegue lê-las. O escopo correto é por step: ANTHROPIC_API_KEY só em fetch.py, GOATCOUNTER_TOKEN só em fetch_stats.py (o if: env.X != '' pode ser resolvido com um step curto que publica um output).

Condição de explorabilidade: Só vira vazamento combinado com execução de código no job (A2).

BAIXA

A4 — CSP com script-src 'unsafe-inline' em todas as páginas

`docs/index.html:8` · `docs/admin/index.html:8` · `docs/embed.html:10` · `demais páginas` · 5. XSS (defesa em profundidade)

```
script-src 'self' 'unsafe-inline' https://gc.zgo.at
```

Por que é explorável: Toda página libera script inline irrestrito, então a CSP não bloqueia injeção de <script> nem URLs javascript: — a proteção anti-XSS fica inteiramente na disciplina do esc() (que hoje é consistente). Como o site usa scripts inline por desenho (sem build), o endurecimento viável é CSP por hash (sha256 dos blocos inline, gerado pelos builders) ou externar os scripts. O guard anti-clickjacking via JS (if top!==self) também é contornável por iframe sandbox — limitação conhecida de site sem headers.

Condição de explorabilidade: Sem efeito isolado; reduz a margem de erro caso um esc() seja esquecido no futuro.

INFORMATIVA

A5 — Painel /admin/ público, protegido só por obscuridade (decisão documentada)

`docs/admin/index.html` · `docs/robots.txt:3` · 2. Permissão no navegador

```
Disallow: /admin/
```

Por que é explorável: O painel de acessos é acessível a qualquer pessoa que conheça a URL — e o robots.txt anuncia o caminho. Não há dado pessoal: só contagens agregadas do GoatCounter, e o docs/data/stats.json que o alimenta já é público de qualquer forma. O próprio HTML documenta a decisão (nota de segurança correta: senha em JS seria teatro) e aponta o caminho certo se um dia precisar de sigilo real. Registrado para visibilidade, sem ação exigida.

Condição de explorabilidade: Aceitável enquanto stats.json contiver apenas agregados.

Recomendações priorizadas

P1	Pinar dependências do CI (versões exatas + pip --require-hashes; pip-compile gera o lock) e escopar os secrets por step em monitor.yml — fecha A2 e A3 de uma vez, sem mudar comportamento.
P2	Antes de ligar o coletor real do Radar: validar esquema de URL em fontesHTML (permitir só http(s), senão renderizar como texto) — fecha A1 no contrato de dados, não na sorte.
P3	Endurecer a CSP das páginas de maior tráfego trocando 'unsafe-inline' por hashes sha256 gerados pelos builders (começar por index.html e embed.html) — reduz A4 gradualmente.
P4	Quando o palpite.html/Supabase entrar: aplicar exatamente as regras já escritas no SECURITY.md (RLS insert-only, service key só no Actions, CHECK no banco) e auditar de novo as categorias 1 e 3, que passarão a existir.

Issues para o GitHub

Texto completo de cada issue em Markdown, pronto para copiar e colar, entre os delimitadores. A2 e A3 foram agrupados numa issue única (mesmo workflow, mesma correção); A5 é decisão documentada e não gera issue.

```
--- ISSUE 1 ---
Título: [Segurança] XSS via URL javascript: nas fontes citadas do recibo do Radar
Labels: security, severity:media

## Problema

`fontesHTML()` em `docs/radar/radar.js` monta os links das fontes citadas com
`esc(f.url)`. O `esc()` escapa `&<>"'` (impede quebrar o atributo), mas não
valida o **esquema** da URL: um valor `javascript:...` vira um link que executa
script no clique. A CSP das páginas do Radar inclui `'unsafe-inline'`, que
também libera navegação `javascript:`.

Hoje os recibos são fixtures commitadas (inofensivas), mas o campo
`fontes_citadas` virá do coletor real, com URLs extraídas de respostas de
modelos de IA — conteúdo de terceiros que pode ser influenciado por quem
otimiza páginas para ser citado.

## Evidência

`docs/radar/radar.js:352`

```js
return '<li><a href="' + esc(f.url) + '" target="_blank" rel="noopener noreferrer nofollow">' +
esc(f.titulo || f.url) + "</a></li>";
```

Impacto

Execução de JavaScript arbitrário no contexto do site para quem clicar numa
fonte de um recibo envenenado (roubo de nada sensível hoje, mas defacement,
phishing e abuso da origem do domínio).

Correção sugerida

Validar o esquema antes de renderizar como link; caso contrário, texto puro:

```js
function urlSegura(u) {
return /^https?:\/\//i.test(String(u || "")) ? u : null;
}
// em fontesHTML():
var u = urlSegura(f.url);
if (!u) return "<li>" + esc(f.titulo || f.url) + "</li>";
```

Critérios de aceite

- [ ] URL com esquema diferente de http/https é renderizada como texto, nunca como href
- [ ] Fixture de teste com `javascript:alert(1)` em fontes_citadas não produz link clicável
- [ ] Contrato de dados do coletor (docs/radar/README.md) documenta a regra
- [ ] Nenhuma regressão nos recibos existentes (fixtures continuam renderizando)
--- FIM ISSUE 1 ---
```

```
--- ISSUE 2 ---
Título: [Segurança] Pinar dependências do CI e escopar secrets por step no monitor.yml
Labels: security, severity:media

## Problema

Dois achados relacionados no mesmo workflow (agrupados para não gerar spam):

1. **Dependências sem pinagem** — `requirements.txt` lista `requests`, `pyyaml`
e `pillow` sem versão nem hash, e o workflow ainda faz `pip install anthropic`
solto. Cada execução do cron (a cada 6 h) instala a última versão publicada
no PyPI. O job tem `permissions: contents: write`: uma release maliciosa
executaria com poder de commitar no repositório.
2. **Secrets com escopo largo** — `ANTHROPIC_API_KEY` e `GOATCOUNTER_TOKEN` são
env do job inteiro; todos os steps (inclusive o `pip install`) conseguem
lê-los, ampliando o impacto do item 1 para vazamento de chaves.

Contrasta com o resto do projeto, que pina Actions por SHA e usa SRI no CDN.
O Dependabot já monitora pip, mas sem versões pinadas não tem o que atualizar.

## Evidência

`requirements.txt:1-3`

```
requests
pyyaml
pillow
```

`.github/workflows/monitor.yml:29-31`

```yaml
env:
ANTHROPIC_API_KEY: ${{ secrets.ANTHROPIC_API_KEY }}
GOATCOUNTER_TOKEN: ${{ secrets.GOATCOUNTER_TOKEN }}
```

`.github/workflows/monitor.yml:44,50` — `pip install -r requirements.txt` /
`pip install anthropic` sem pin.

Impacto

Comprometimento de pacote PyPI (supply chain) vira execução de código com
escrita no repo + leitura das duas chaves. Probabilidade baixa, impacto alto.

Correção sugerida

- Pinar versões exatas e hashes: `pip-compile --generate-hashes` e
`pip install --require-hashes -r requirements.txt` (o Dependabot passa a
propor os bumps).
- Pinar `anthropic==X.Y.Z` no mesmo lock (extra opcional).
- Mover cada secret para o env do único step que o usa; para os `if:`, um step
inicial pode publicar `has_anthropic`/`has_goat` como outputs.

Critérios de aceite

- [ ] requirements.txt (ou lock derivado) com versão exata + hash de todas as deps
- [ ] Workflow instala com --require-hashes e falha se um hash divergir
- [ ] ANTHROPIC_API_KEY visível apenas no step "Coletar séries" (e no install opcional)
- [ ] GOATCOUNTER_TOKEN visível apenas no step "Coletar estatísticas"
- [ ] Cron continua verde por 2 execuções seguidas após a mudança
--- FIM ISSUE 2 ---
```

```
--- ISSUE 3 ---
Título: [Segurança] Endurecer a CSP: substituir 'unsafe-inline' por hashes nos scripts inline
Labels: security, severity:baixa

## Problema

Todas as páginas publicam CSP via `<meta>` com
`script-src 'self' 'unsafe-inline' ...`. Com `'unsafe-inline'`, a CSP não
bloqueia `<script>` injetado nem URLs `javascript:` — a defesa anti-XSS fica
inteiramente no `esc()` (hoje consistente, mas sem rede de segurança para um
esquecimento futuro). Como o site usa scripts inline por desenho (sem build,
legível via file://), o endurecimento viável é CSP por hash.

## Evidência

`docs/index.html:8` (padrão repetido em todas as páginas)

```
script-src 'self' 'unsafe-inline' https://gc.zgo.at
```

Impacto

Nenhum isoladamente — é defesa em profundidade. Se um novo ponto de
interpolação esquecer o `esc()`, a CSP atual não segura nada.

Correção sugerida

- Gerar `sha256-...` dos blocos `<script>` inline nos builders
(`build_pages.py`, `build_correcao.py`, `render_index.py`) e escrever
`script-src 'self' 'sha256-...' https://gc.zgo.at`.
- Começar pelas páginas de maior tráfego (index.html, embed.html) e expandir.
- Manter `'unsafe-inline'` apenas em style-src (inofensivo em comparação).

Critérios de aceite

- [ ] index.html e embed.html sem 'unsafe-inline' em script-src, com hashes válidos
- [ ] Páginas carregam sem erro de CSP no console (verificado nos dois temas do embed)
- [ ] Builders regeneram os hashes automaticamente quando o script inline muda
- [ ] SECURITY.md atualizado descrevendo a regra nova
--- FIM ISSUE 3 ---
```